

Founder Intelligence — Execution Blueprint

Prepared for: Manan Panchal (Project Manager) · **Team size:** 26
Source: FI Full Product Bible, Neat Edition, 2026 (DEZAI Technologies × The First Brick Community)
Status: v1 — team groupings are a starting proposal, explicitly designed to be reshuffled

1. How to read this document

The Bible (47 sections) is the *what* and *why*. This document is the *how*: who builds it, in what order, on what stack, and with what architecture, at two horizons — an **MVP** you can put in front of real founders in Dubai/UAE and India within weeks, and the **Production** system the Bible ultimately describes (§28–34, §40–43).

Everything here has a matching, runnable scaffold in this repository (`apps/` , `packages/` , `db/` , `infra/` , `.github/`) — this is not a slideware plan, it’s the plan the folder structure implements.

2. Team structure (26 people 9 squads)

The Bible’s system naturally splits into five *layers* (data/schema, AI research engine, decision logic, API, UI) plus delivery and platform concerns. I’ve mapped your roster onto that split. Roles you gave me (Full stack, Backend, AI/ML, Frontend, UI/UX, DevOps, PM) map directly; three “Full stack Developer” hires with no stated specialty are placed as a flex pod that takes on whichever squad is behind schedule — that’s the group to reshuffle first.

#	Squad	Mission	Lead	Members
1	PMO & Delivery	Scope, sequencing, standups, unblocking, stakeholder comms	Manan (PM)	Harshil Thakkar (Team Manager — pace/blockers)
2	Platform / DevOps / QA	CI/CD, envs, infra, observability, cost, cross-cutting test strategy	Rishi Bharatbhai Sanja (Full stack + DevOps)	Rahil (Cloud intern, 4mo), Hitarth (role TBC — good fit for QA)
3	Core Backend (<code>apps/api</code>)	Auth, workspaces, ideas, CRUD, billing, exports, SSE gateway — the system of record	Mann Shah (Backend)	Krish Parmar, Nil, Deep, Tirth Bhanderi

4	AI — Research Engine (apps/ai/research)	Scope Plan Discover Fetch Extract pipeline (Bible §9)	Sharanam Katwala (AI/ML)	Preet Savaliya, Jiya Sadaria (AIML Backend), Vraj (intern, 6mo)
5	AI — Reasoning, Scoring & Evals	Crosscheck/Synthesize/Recommend, opportunity score, evals, prompt registry	Shreeji Sojitra (AI/ML — interview reliability)	Yug Bhatt
6	Frontend Core (apps/web)	App shell, auth flows, state, data layer, all screens' scaffolding	Aanshi Bhut (Frontend core flow, App state)	Varun, Prashant
7	Product Design	Design tokens, IA, the “premium intelligence instrument” visual system (§32)	Akanksha (UI/UX)	Krish Singh (Product design + backend swing)
8	Feature Pod A — Decision Layer	Assumptions, Experiments, Decision Trace, Scorecard UI+API	Ansh Dhanani (Full stack)	Avi Patel
9	Feature Pod B — Reports, Workspace, Billing	Report system, exports, workspace/collab, pricing tiers	Krish Bhingradiya (Full stack)	Anmol Dholiya
—	Feature Pod C — Radar & Intelligence Views	Market/Competitor/Customer screens, Radar UI	Romit Kakadiya (Full stack)	borrow from Pod A/B once Phase 2 lands

Reshuffle rule of thumb: keep squads 3, 4, 5 stable — they own hard, load-bearing internals (schema, pipeline, scoring) where context-switching is expensive. Rotate people through squads 6, 8, 9, C freely; those are feature pods against a contract (packages/contracts) and a design system (packages/ui), so swapping people costs little once both are stable.

Note: three roster entries (Yug Bhatt, Hitarth) have no confirmed handle/role — confirm before sprint 1 so CODEOWNERS and standup rotations are accurate. Three people share the first name “Krish” (Parmar, Singh, Bhingradiya) — always tag by full name/handle in issues.

3. Two build horizons

	MVP (Private alpha Founder beta, Bible §39 stages 1–2)	Production (Public launch Team expansion, §39 stages 3–5)
Goal	Prove research quality + repeat usage with real founders	Prove retention, scale, defensibility (§40)
Modules live	Idea Intake, Adaptive Interview, Research Engine (quick+standard depth), Evidence, Market/Customer/Competitive/Pricing/GTM intelligence, Opportunity Score, Assumptions, basic Reports	+ Experiment Lab, Decision Trace, Founder Copilot, Continuous Radar, Investor Mode, Workspace collaboration, Billing, deep-research depth
Users	Invite-only, single workspace per founder	Multi-tenant, teams, incubator/enterprise accounts
Infra	Managed PaaS (fast, cheap, low ops)	Own VPC, IaC, multi-region-ready (UAE + India)
Timeline	~8–10 weeks	Rolling, phased over the roadmap in §9

4. Technology stack

4.1 MVP stack — optimize for speed and low ops burden

Layer	Choice	Why
Frontend	Next.js 15 (App Router) + React 18 + TypeScript , deployed on Vercel	Team already skews full-stack JS; Vercel = zero-config CI/CD, preview URLs per PR (huge for a 26-person team reviewing UI)
Design system	Plain CSS variables + component lib in <code>packages/ui</code> (Bible §32 tokens)	No heavy UI framework lock-in yet; Akanksha's design tokens are the actual system
Core API	Node.js 22 + Express 5 + TypeScript , deployed on Render/Railway	Matches your Backend squad's stated skills (Express routes, env,

		health — Mann Shah's own description)
AI service	Python 3.12 + FastAPI + Celery , deployed as a second Render/Railway service	AI/ML squad works in Python natively; Celery gives you a real job queue for long research runs without building one
Database	Postgres 16 with pgvector , hosted on Supabase	One managed service gives you Postgres + pgvector (embeddings) + Auth + Storage + Row-Level-Security — huge MVP leverage
Auth	Supabase Auth (email + OAuth), JWT verified in Express via <code>jose</code>	No separate auth service to build/run at MVP stage
Queue/cache	Upstash Redis (serverless)	Celery broker + SSE pub/sub, no server to manage
Search/retrieval	Tavily / Exa (LLM-native search APIs) + <code>trafilatura</code> for readability extraction	Purpose-built for AI research agents; avoids building a crawler at MVP stage
LLM	Model router (<code>apps/ai/src/fi_ai/llm/router.py</code>) over a fast/cheap tier + a strong reasoning tier (Bible §31) — provider chosen at build time, never hardcoded per call site	Keeps you provider-agnostic and cost-controlled from day one
Object storage	Supabase Storage (source snapshots, exports, uploaded interview transcripts)	Same platform, no extra vendor
Observability	Sentry (errors), Langfuse (LLM tracing/cost), PostHog (product analytics)	All have generous free tiers; Langfuse is essential given §31/§41's cost-per-project KPI
Exports	Playwright PDF , <code>docx</code> npm lib for Word	Cheapest way to hit the \$25 export targets
CI/CD	GitHub Actions (already scaffolded in <code>.github/workflows/ci.yml</code>)	Free, matches monorepo

4.2 Production stack — optimize for control, scale, and the UAE+India roadmap (\$40)

Layer	Choice	Why
Frontend	Same Next.js app, deployed via containerized ECS/Fargate + CloudFront , or stay on Vercel Enterprise if that's operationally preferred	Bible doesn't force a migration; decide based on your actual scaling pain, not on principle
Core API	Same Express service, containerized, run on ECS Fargate (or GKE if the team prefers Kubernetes) behind an ALB	Horizontal scaling, blue/green deploys, VPC isolation
AI service	Same FastAPI app; Celery workers become an autoscaled worker pool (scale on queue depth) separate from the API pods	Research runs are bursty and long — decouple scaling from the request-serving API
Database	Amazon RDS for Postgres (Multi-AZ) with pgvector , one cluster per region (me-central-1 UAE, ap-south-1 India) per \$40's "geography as a first-class parameter"	Data residency for GCC/India compliance, lower latency, disaster recovery
Search index	Add OpenSearch/Elasticsearch for evidence/claim full-text + hybrid search once corpus size makes pgvector-only search slow	Only needed once you have real research-history volume (\$26, moat data)
Queue	Amazon SQS or self-hosted Redis cluster (ElastiCache)	Managed, HA, matches AWS-native infra
Secrets	AWS Secrets Manager	Rotate LLM/search API keys without redeploy
IaC	Terraform , modules scaffolded in infra/terraform	Reproducible envs per region, matches your DevOps squad's stated focus (deploy hygiene)
Networking/security	VPC + WAF + CloudFront , per-tenant rate limiting at the edge	Needed once you have paying/enterprise customers (\$26 governance)
Model strategy	Same router, but add self-hosted or reserved-throughput inference for the	This is the single biggest lever on research cost per project

	fast/extraction tier if volume makes per-call API pricing expensive (\$31 cost KPI)	
Observability	Add OpenTelemetry traces across Node Python DB, Grafana/Prometheus dashboards, PagerDuty on-call	Multi-service production needs real SRE tooling, not just Sentry
Compliance	Workspace-level encryption at rest, audit log (already in schema as <code>activity_log</code>), data export/delete flows (\$27, \$36 Enterprise tier)	Required before selling to incubators/enterprises

Migration path is intentionally boring: almost nothing about the *code* changes between MVP and Production — same Express app, same FastAPI app, same schema. What changes is *where it runs* and *how it scales*. That's why the repo already has both `infra/render.yaml` (MVP) and `infra/terraform/` (Production) pointed at the same Dockerfiles.

5. High-Level Design (HLD)



Search/Fetch LLM Router
Tavily/Exa + fast/strong
trafilatura tier models

Redis (Celery broker
+ SSE pub/sub)

Why two backend runtimes instead of one (ADR-0002): Core API is CRUD/authZ/billing-shaped —

Express is the team's stated strength and is the wrong tool for a multi-stage LLM pipeline with retries, long-running jobs and evals. The AI service is pipeline/agent-shaped — Python has the ecosystem (litellm, trafilatura, embeddings tooling) and is where your AI/ML squad already thinks. They share one Postgres as the single source of truth, so there's no data-sync problem, only an API/job contract (`packages/contracts` + the internal HTTP calls in `apps/api/src/lib/queue.ts` and `apps/ai/src/fi_ai/main.py`).

Request flow example — “Start research” (Bible §6 steps 5–8):

1. Browser `POST /v1/workspaces/:id/ideas/:id/research/runs` (Express, `research.routes.ts`)
 2. Express validates against `packages/contracts` , checks workspace membership, checks a `research_brief` exists, inserts a `research_runs` row (`state=created`), then calls the AI service's internal `/internal/research/runs/:id/start` .
 3. AI service enqueues `research.run` on Celery and returns `202` immediately — Express returns `202` to the browser.
 4. Browser opens `GET /v1/.../runs/:runId/events` (SSE) — Express subscribes to Redis channel `run:{id}` and streams events straight through.
 5. Celery worker's `Orchestrator` walks the state machine in `research/state.py` (`scoping` `planning` `discovering` `fetching` `extracting` `crosschecking` `synthesizing` `recommending` `completed`), publishing a progress event after every stage, and writing `sources` / `evidence` / `claims` / `insights` rows as it goes.
 6. On completion, Reasoning squad's scoring step (`packages/scoring`) computes the opportunity score from dimension inputs the AI service produced; Express (or a Python endpoint — pick one owner, don't duplicate) persists `opportunity_scores` .
 7. Browser's Intelligence Home / Market Map / Scorecard screens read the now-populated tables through Express `GET` endpoints.
-

6. Low-Level Design (LLD) — the two hardest subsystems

6.1 Evidence & claim-status engine (Bible §10, §27)

This is FI's actual product differentiator, so it's built as **deterministic code that consumes LLM output**, not as an LLM decision. Implemented in `apps/ai/src/fi_ai/evidence/status.py` :

```
derive_claim_status(origin, evidence[], kind, ttl_days, relevance_min) -> ClaimStatus
```

Rules encoded (already unit-tested in `apps/ai/tests/test_status.py`):

- `evidence` supporting AND contradicting `conflicting`
- No qualifying evidence, but the claim came from the founder or is an FI hypothesis `assumption`
- No qualifying evidence at all `unverified`
- `origin == "inferred"` `inferred` (always, regardless of evidence — it inherits its parents' caveats)
- A `signal` -kind claim (search interest, discussion volume) can never rise above `directional` , per Appendix A: "do not confuse a signal with proof"
- `fact` -kind claim backed by a **fresh, primary** source (official site, government, filing, or user upload, within `ttl_days`) `verified`
- `fact` -kind claim backed by **2+ independent publishers** `supported`
- Otherwise `directional`

Every `Claim` row stores `evidence` links (`claim_evidence` join table) and `claim_contradictions` , so the UI's Evidence screen and Decision Trace can render the *why* behind every badge — not just the badge.

Quality gate (`evidence/quality_gate.py`) runs before any `insight` or `recommendation` is persisted: it fails closed if a claim cites no evidence, cites a non-existent claim ID, or omits an unknowns section — directly implementing §27's "quality gate" checklist.

6.2 Research orchestrator (Bible §9)

`apps/ai/src/fi_ai/research/orchestrator.py` implements the 8-stage pipeline as an explicit state machine (`research/state.py`), not one giant prompt — this was an explicit Bible requirement ("A strong research engine must also know when not to make a claim... FI should use a multi-stage research pipeline rather than a single giant prompt").

Design decisions baked into the LLD:

- **Idempotent stages:** every stage writes with natural keys (e.g. `sources` unique on `(url, content_hash)`) so a retried/resumed run never double-writes.
- **Budget enforcement:** `RunContext.over_budget()` checked before every stage transition — a `quick` run has a hard USD ceiling (`RUN_BUDGET_USD_QUICK` etc.), and exceeding it

produces a

partial state with whatever evidence was gathered, rather than failing silently or overspending.

- **Progress events:** every stage transition publishes to Redis (`run:{id}`) — this is the same channel Express subscribes to for the SSE endpoint, so “Research Live” (§33) needs no polling.
 - **Extraction hard gate:** `extract_evidence.v1.yaml` prompt requires quotes to be verbatim substrings of the fetched snapshot; anything that fails that check is discarded before it ever becomes a `Claim` — this is the main lever on the `unsupported_claim_rate` KPI (§41).
 - **Model routing:** every LLM call goes through `llm/router.py` by *task name* (`extract_evidence` , `synthesize_module` , ...), which maps to a fast/strong tier in `llm/models.yaml` . Changing providers or rebalancing cost/quality is a one-file YAML change, never a code change (§31).
-

7. Data model summary

Full DDL is in `db/migrations/001...005_*.sql` . Entity groups map directly to Bible §29:

- **Tenancy:** `users` , `workspaces` , `workspace_members` , `ideas` (Row-Level Security enabled as defense-in-depth alongside app-layer checks)
 - **Interview:** `research_briefs` , `interview_turns`
 - **Research pipeline:** `research_runs` , `sources` , `evidence` (with `pgvector` embedding column), `claims` , `claim_evidence` , `claim_contradictions` , `insights`
 - **Decision layer:** `opportunity_scores` , `assumptions` , `experiments` , `recommendations` , `decisions`
 - **Radar/Reports/Billing:** `watch_items` , `alerts` , `reports` , `subscriptions` , `activity_log`
 - **Cost/observability:** `llm_calls` (per-call token/cost/latency — feeds the §41 cost KPI directly)
-

8. API surface (v1) — see `packages/contracts` for the typed source of truth

```
POST /v1/workspaces
GET /v1/workspaces
GET /v1/workspaces/:workspaceId/ideas
POST /v1/workspaces/:workspaceId/ideas
GET /v1/workspaces/:workspaceId/ideas/:ideaId
POST /v1/workspaces/:workspaceId/ideas/:ideaId/research/runs
GET /v1/workspaces/:workspaceId/ideas/:ideaId/research/runs/:runId/events (SSE)
```

scaffolded as module stubs, ready for each owning squad to fill in (see apps/api/src/modules/*/interview · evidence · competitors · customers · scorecard · assumptions · experiments · decisions · copilot · radar · reports · billing · notifications

Internal (service-to-service, `x-internal-token` header, not public):

POST /internal/research/runs/:runId/start (Express -> AI service)
POST /internal/copilot/ask (planned)
POST /internal/radar/check (planned, called by scheduler)

9. Roadmap & squad assignment by phase

Mirrors Bible §42/§39, with squads from §2 assigned:

Phase	Weeks (indicative)	Capabilities	Primary squads
0. Foundation	1–2	Repo (done), auth, workspace, idea model, schema, CI, envs	Platform/DevOps, Core Backend
1. MVP Research	3–6	Adaptive interview, research engine (quick/standard), market+customer+competitor+pricing+GTM synthesis, reports v1	AI-Research, AI-Reasoning, Core Backend, Frontend Core
2. Validation	6–8	Opportunity score, assumptions, risk register, recommendations, experiment lab	AI-Reasoning, Feature Pod A
3. Intelligence	8–10	Evidence graph UI, decision trace, Founder Copilot, research history	AI-Reasoning, Feature Pod A, Frontend Core
4. Radar	10–13	Competitor/customer/demand/tech/regulatory watch + alerts	AI-Research, Feature Pod C
5. Primary research	13–16	Interview upload/transcript analysis, survey workflows	AI-Research, Feature Pod B
6. Team/Enterprise	16–20	Collaboration, roles, governance, billing tiers, SSO	Core Backend, Feature Pod B, Platform
7. Proprietary intelligence	20+	Benchmarks, vertical packs, outcome-informed scoring	AI-Reasoning (data science extension)

MVP = Phases 0–2 roughly. **“Production-hardened”** = once Phase 3 ships and you move infra from §4.1 to §4.2.

10. Working agreements (so 26 people don’t collide)

- **Trunk-based development.** Short branches, PRs $\leq$ ~400 lines, merge to `main` daily.
 - **Contract-first.** Any API/event shape change starts in `packages/contracts` ; both Node and Python sides regenerate/validate against it in CI.
 - **One schema source of truth.** All schema changes are a new file in `db/migrations/` ; no ORM
auto-migrations, no editing merged migrations (ADR-0003).
 - **Prompts are code.** Every prompt lives in `apps/ai/src/fi_ai/prompts/*.yaml` , versioned, requires
2 reviewers + a passing `make eval` (CI enforces this on any path touching `prompts/`).
This is the single most important discipline for hitting the §41 `unsupported_claim_rate` and `freshness_coverage` KPIs — it’s also usually where teams get sloppy first, so protect it deliberately.
 - **Scoring is deterministic, never LLM-generated directly** (ADR-0004) — LLMs propose, `packages/scoring` computes.
 - **CODEOWNERS** (`.github/CODEOWNERS`) map paths `squads`, not individuals, so reshuffling people
is a GitHub team-membership change, never a file-ownership rewrite.
 - **Definition of done** for any module: contract updated migration (if needed)
implementation
tests eval (if AI) wired into `app.ts` / `main.py` screen consumes it PR includes screenshots/curl examples.
-

11. What’s already runnable in this repo vs what’s a stub

Runnable / real logic today:

- Full monorepo tooling (pnpm/turbo, uv, Docker, CI, migrations)
- `packages/contracts` — zod schemas mirroring the Python pydantic schemas
- `packages/scoring` — real, tested opportunity-score and experiment-ranking math
- `apps/ai/.../evidence/status.py` — real, tested claim-status derivation (this is the core IP)
- `apps/ai/.../research/state.py` + `orchestrator.py` — real state machine, stage registry, budget
guard, event publishing (the LLM calls and DB I/O inside each stage are stubs — see below)
- `apps/api` — real Express app: auth middleware, workspace authorization, `ideas` module fully
wired end-to-end as the pattern to copy, `research` run-creation + SSE streaming wired to the
AI service

- `apps/web` — real Next.js app shell with every Bible §33 screen present as a routed page, ready for each squad to fill in
- Full SQL schema for the entire Bible's data model (`db/migrations`)

Intentional stubs (marked `TODO(<squad>)` in code) — this is where sprint 1 work starts:

- The 8 pipeline stage functions in `apps/ai/.../research/pipeline/*.py` (search/fetch/extract/etc.)
- `llm/router.py` 's actual `litellm` call + retry/validation logic
- Remaining `apps/api/src/modules/*` (interview, evidence, competitors, ...) — copy the `ideas` module pattern
- Founder Copilot retrieval, Radar diffing, report rendering, billing webhooks

This split is deliberate: the scaffold encodes every *structural* decision from the Bible (schema, state machine, evidence rules, contracts, team boundaries) so that Sprint 1 is pure feature work, not architecture debate.